

머신러닝

k-최근접 이웃

Chapter 11



Prof. Yang



한국성서대학교
KOREAN BIBLE UNIVERSITY

목 차

CONTENTS

- 01 k-최근접 이웃의 개념
- 02 k-최근접 이웃을 활용한 분류
- 03 k-최근접 이웃을 활용한 회귀
- 04 k-최근접 이웃의 하이퍼파라미터

01

k-최근접 이웃의 개념



모델 기반 학습

- 훈련 데이터로부터 모델 f 의 파라미터(가중치)를 최적화하여 최적의 함수 f^* 생성
- 한 번 모델 생성을 완료하면 테스트 데이터에 대해서는 단순히 함수 f^* 의 연산으로 예측값 도출
- 예) 선형회귀, 로지스틱회귀, 나이브베이즈안 등 앞서 배운 모델이 모두 모델 기반 학습 방법

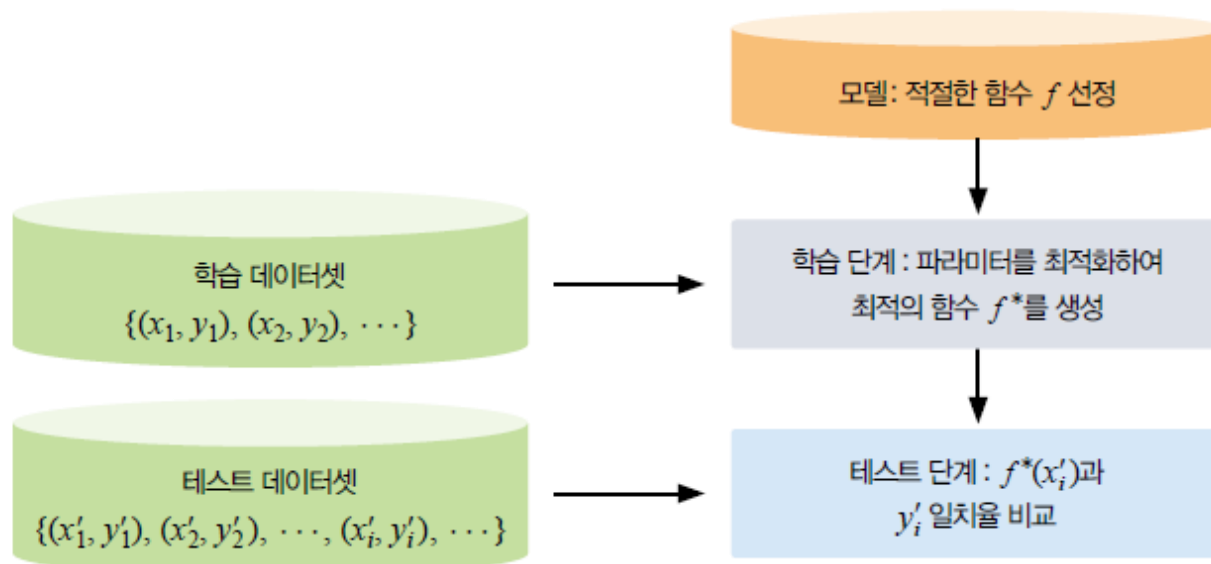


그림 7-1 모델 기반 학습 예



사례 기반 학습

- 별도의 모델을 생성하지 않는 방식
- 테스트 데이터가 입력될 때마다 학습 데이터를 기반으로 예측값 연산
- k-최근접 이웃이 대표적인 사례 기반 학습 방법임

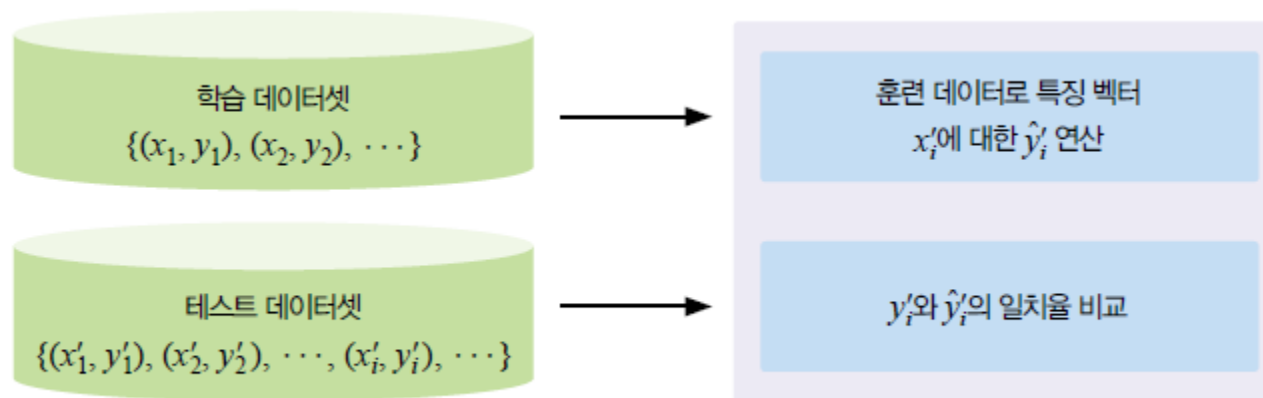


그림 7-2 사례 기반 학습 예

K-최근접 이웃(k-Nearest Neighbor, KNN)

: 지도학습 방법의 하나로, 새로운 데이터를 기존 데이터와 비교하여 분류하는 알고리즘

- k: 근접해 있는 데이터 중 몇 개를 확인할 것인지를 의미
 - k가 1보다 크다면 주변 데이터의 이산형 실제값에 따라 다수결로 새로운 이산형 실제값을 결정

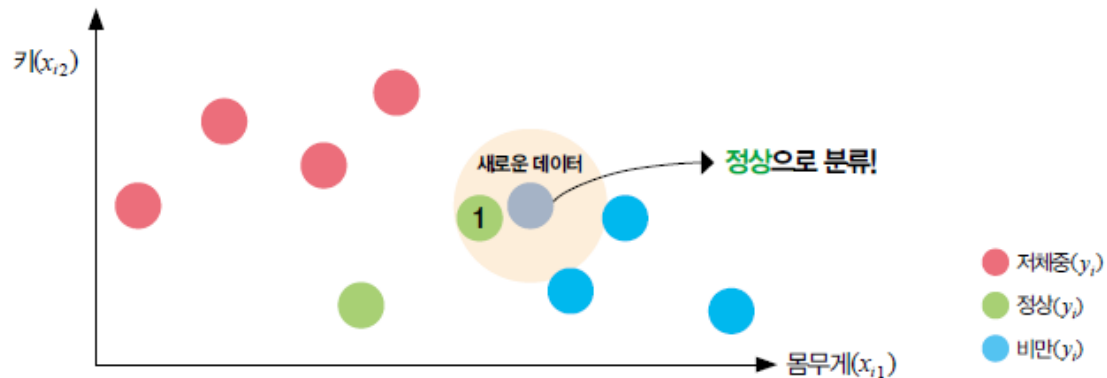


그림 7-4 k-최근접 이웃 예 (k=1)

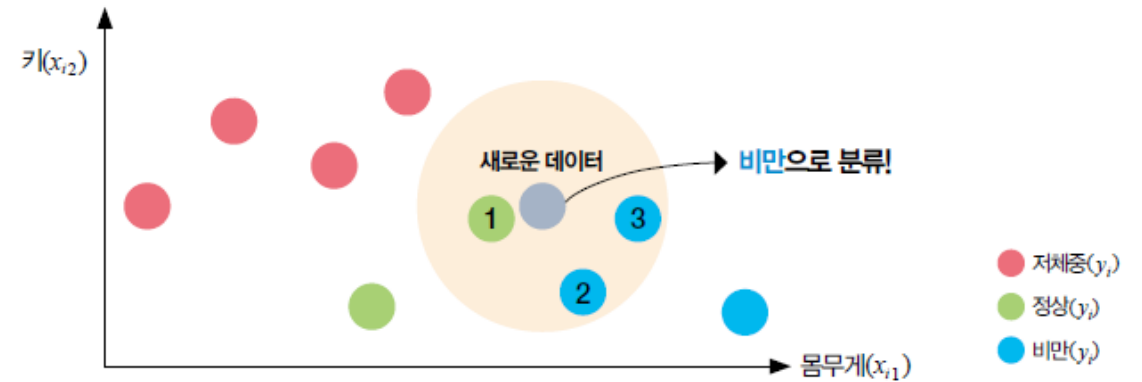


그림 7-5 k-최근접 이웃 예 (k=3)



거리 측정

- 테이블 데이터에서는 근접한 데이터를 어떻게 계산할까? → 거리 측정
- 새로운 데이터가 발생한 이후에 실제값 예측 수행됨 → 사례 기반 학습의 특징
 - 아래 데이터 간 유클리드 거리와 맨해튼 거리를 구함

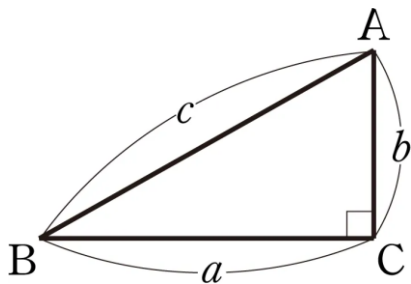
표 7-1 키-몸무게 등급에 따른 비만 여부

	몸무게(x_{i1})	키(x_{i2})	비만 여부(y_i)
생도 1	1	0	0
생도 2	2	1	0
생도 3	3	3	0
생도 4	5	2	1
생도 5	5	4	1
생도 6	6	5	1
생도 7	9	2	1
신입 생도	3	4	?



유클리드 거리

- 가장 많이 사용하는 거리측도 → 고등수학 과정의 직선 거리, 점과 점 사이의 거리
- 대응되는 관측치의 특징벡터 간 차이 제곱 합의 제곱근
- 기본적으로 이 계산을 많이 사용함



$$a^2 + b^2 = c^2$$

$$\text{Euclidean Distance}(\mathbf{x}_1, \mathbf{x}_2) = \sqrt{\sum_{j=1}^p (x_{1j} - x_{2j})^2}$$

• p : 데이터(특징 벡터)를 구성하는 성분의 수

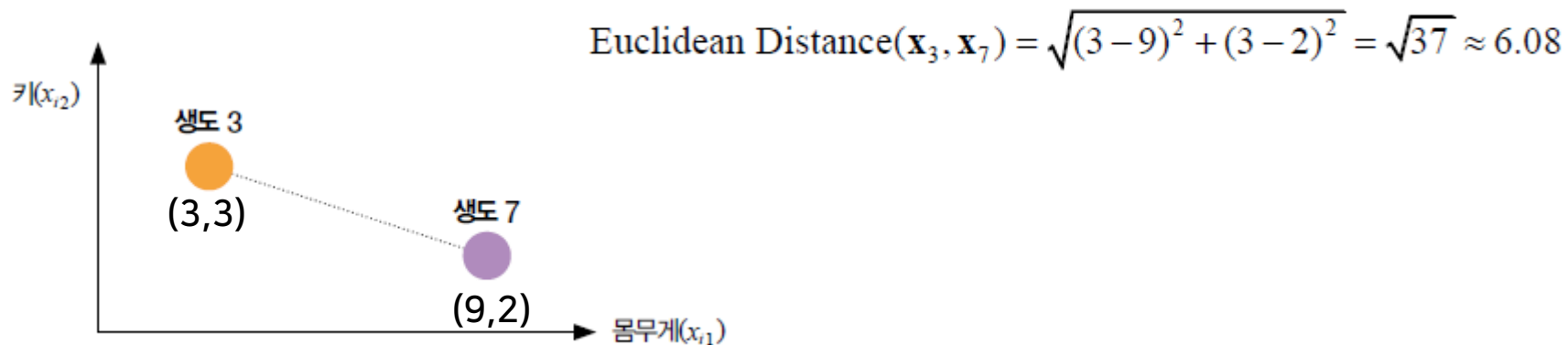


그림 7-6 유클리드 거리 예



맨해튼 거리

- 일상생활에서는 잘 사용하지 않으나, 머신러닝 혹은 프로그래밍에서 종종 사용
- 실제 뉴욕의 맨해튼과 같은 구획에서 이동할 때는 맨해튼 거리처럼 이동해야 함



그림 7-7 맨해튼 지도

$$\text{Manhattan Distance}(\mathbf{x}_1, \mathbf{x}_2) = \sum_{j=1}^p |x_{1j} - x_{2j}|$$

• p : 데이터(특징 벡터)를 구성하는 성분의 수

$$\text{Manhattan Distance}(\mathbf{x}_3, \mathbf{x}_7) = |3 - 9| + |3 - 2| = 7$$

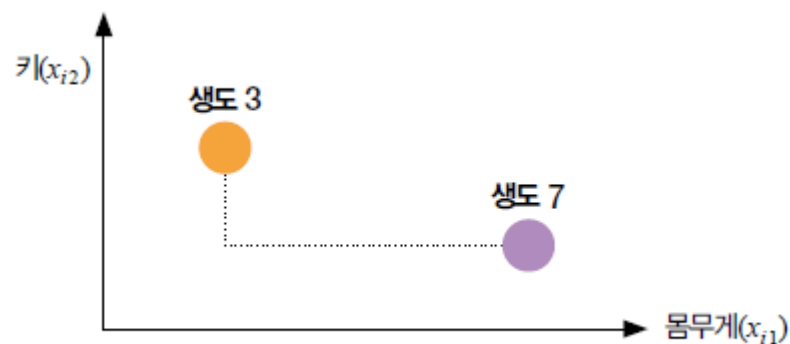


그림 7-8 맨해튼 거리 예

02

k-최근접 이웃을 통한 분류



k-최근접 이웃을 통한 분류

- k-최근접 이웃 동작 단계: 새로운 데이터의 예측값을 결정하기 위한 3단계 과정
 - 1) 1단계: 거리 계산
 - 모든 학습 데이터와 새로운 데이터 사이의 거리 계산, 유클리드 거리(기본값) 사용
 - 2) 2단계: 가까운 데이터 탐색
 - k 값에 따라 가까운 데이터 탐색(예시: k=3)

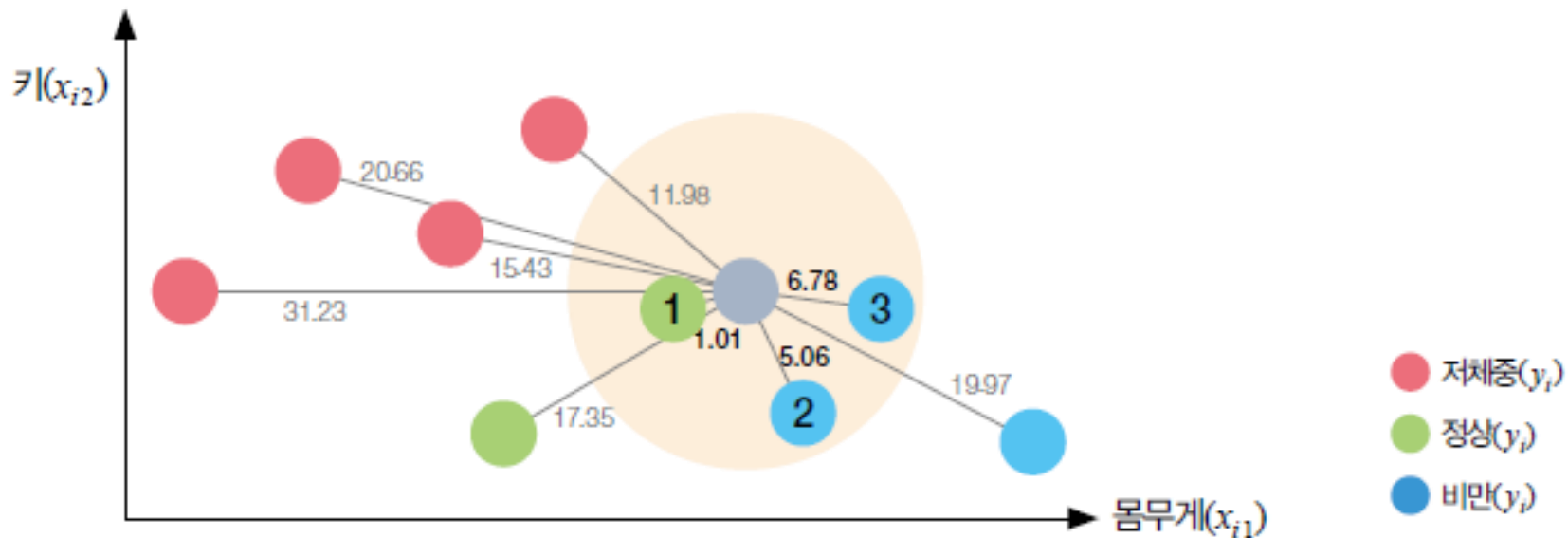


그림 7-9 k-최근접 이웃을 통한 분류 예 (k=3): 1, 2단계



k-최근접 이웃을 통한 분류

- k-최근접 이웃 동작 단계: 새로운 데이터의 예측값을 결정하기 위한 3단계 과정

3) 3단계: 예측값 결정

- k개의 데이터에서 가장 많이 등장한 이산형 실제값으로 예측

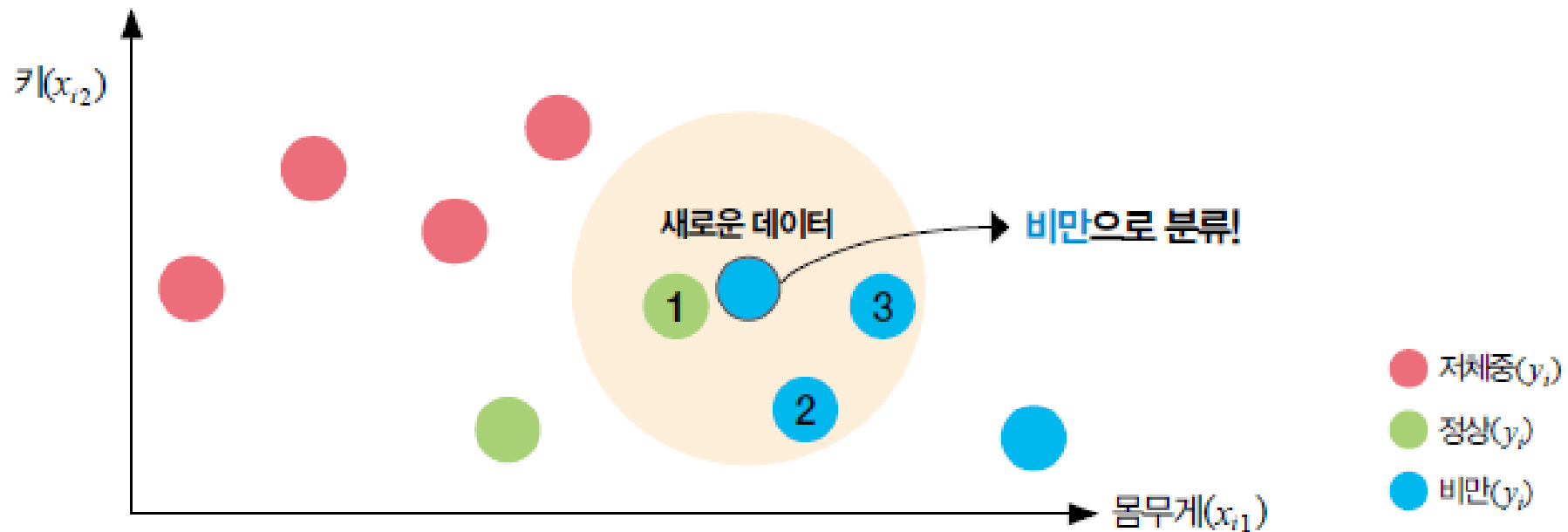


그림 7-10 k-최근접 이웃을 통한 분류 예 ($k=3$): 3단계



k-최근접 이웃을 통한 분류 코드

- 환자의 유전자 정보와 코로나 확진 여부를 k-최근접 이웃 문제를 통해 예측

표 7-2 유전자 정보에 따른 코로나 확진 여부 데이터(분류)

	유전자 1	유전자 2	유전자 3	유전자 4	확진 여부(y_i)
환자0	2.54	4.33	3.99	2.57	정상
환자1	3.12	3.87	3.84	3.04	정상
환자2	2.76	4.17	5.63	3.28	정상
환자3	3.87	3.56	4.25	3.65	확진
환자4	3.55	3.91	2.68	4.22	확진
환자5	4.12	2.86	3.30	3.71	확진
새로운 환자	3.24	3.68	3.82	3.77	?



데이터 준비

```
import numpy as np

# 유전자 정보 4개를 가진 학습 데이터
X = np.array([[2.54, 4.33, 3.99, 2.57], # 환자 0
              [3.12, 3.87, 3.84, 3.04], # 환자 1
              [2.76, 4.17, 5.63, 3.28], # 환자 2
              [3.87, 3.56, 4.25, 3.65], # 환자 3
              [3.55, 3.91, 2.68, 4.22], # 환자 4
              [4.12, 2.86, 3.30, 3.71]]) # 환자 5

# 확진 여부
labels = np.array(["정상", "정상", "정상", "확진", "확진", "확진"])

# 신입 환자 데이터
new = np.array([[3.24, 3.68, 3.82, 3.77]])
```



k=1인 경우 학습하기: 유클리드 거리 계산 기반

- 환자 A~F와 신입환자의 유전자 정보를 통해 거리를 연산
- 가장 가까운 데이터의 이산형 실제값을 통해 새로운 환자의 확진 여부를 결정함 (k=1)

표 7-3 유전자 정보에 따른 코로나 확진 여부 데이터(k=1)

	유전자 1	유전자 1	유전자 2	유전자 1	확진 여부	새 관측치와의 거리
환자0	2.54	4.33	3.99	2.57	정상	1.54
환자1	3.12	3.87	3.94	3.04	정상	0.76
환자2	2.76	4.17	5.63	3.28	정상	2.00
환자3	3.87	3.56	4.25	3.65	확진	0.78
환자4	3.55	3.91	2.68	4.22	확진	1.28
환자5	4.12	2.86	3.30	3.71	확진	1.31
새로운 환자	3.24	3.68	3.82	3.77	정상	

```
distances = np.sqrt(np.sum((X - new) ** 2, axis=1))
print(distances)
```

[1.54317854 0.76406806 1.99667223 0.78140898 1.28495136 1.31179267]

```
idx = np.argmin(distances)

print("\n가장 가까운 환자:", idx)
print("예측 결과:", labels[idx])
```

가장 가까운 환자: 1
예측 결과: 정상



k=1인 경우 학습하기: 사이킷런 기반

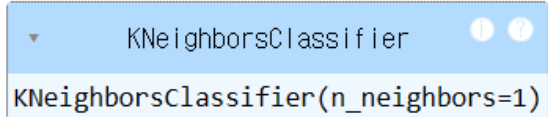
- 환자 A~F와 신입환자의 유전자 정보를 통해 거리를 연산
- 가장 가까운 데이터의 이산형 실제값을 통해 새로운 환자의 확진 여부를 결정함 (k=1)

표 7-3 유전자 정보에 따른 코로나 확진 여부 데이터(k=1)

	유전자 1	유전자 1	유전자 2	유전자 1	확진 여부	새 관측치와의 거리
환자0	2.54	4.33	3.99	2.57	정상	1.54
환자1	3.12	3.87	3.94	3.04	정상	0.76
환자2	2.76	4.17	5.63	3.28	정상	2.00
환자3	3.87	3.56	4.25	3.65	확진	0.78
환자4	3.55	3.91	2.68	4.22	확진	1.28
환자5	4.12	2.86	3.30	3.71	확진	1.31
새로운 환자	3.24	3.68	3.82	3.77	정상	

```
from sklearn.neighbors import KNeighborsClassifier

knr = KNeighborsClassifier(n_neighbors=1) #객체 생성
knr.fit(X, labels)#훈련
```



```
knr.predict(new)

array(['정상'], dtype='<U2')
```


▶ k=3인 경우 학습하기: 유클리드 거리 계산 기반

- 가장 가까운 3개의 데이터의 이산형 실제값을 통해 신입환자의 확진 여부를 결정(k=3)

표 7-4 유전자 정보에 따른 코로나 확진 여부 데이터(k=3)

	유전자 1	유전자 2	유전자 3	유전자 4	확진 여부	새 관측치와의 거리
환자A	2.54	4.33	3.99	2.57	정상	1.54
환자B	3.12	3.87	3.84	3.04	정상	0.76
환자C	2.76	4.17	5.63	3.28	정상	2.00
환자D	3.87	3.56	4.25	3.65	확진	0.78
환자E	3.55	3.91	2.68	4.22	확진	1.28
환자F	4.12	2.86	3.30	3.71	확진	1.31
새로운 환자	3.24	3.68	3.82	3.77	확진	

k = 3

가장 가까운 k개 인덱스

```
idx = np.argsort(distances)[:k]
```

```
votes = labels[idx]
```

```
print(votes)
```

```
if (np.sum(votes == "확진") > np.sum(votes == "정상")): print("확진")
```

```
else: print("정상")
```

→ ['정상' '확진' '확진']
확진



k=3인경우 학습하기: 사이킷런 기반

- 가장 가까운 3개의 데이터의 이산형 실제값을 통해 신입환자의 확진 여부를 결정(k=3)

표 7-4 유전자 정보에 따른 코로나 확진 여부 데이터(k=3)

	유전자 1	유전자 2	유전자 3	유전자 4	확진 여부	새 관측치와의 거리
환자A	2.54	4.33	3.99	2.57	정상	1.54
환자B	3.12	3.87	3.84	3.04	정상	0.76
환자C	2.76	4.17	5.63	3.28	정상	2.00
환자D	3.87	3.56	4.25	3.65	확진	0.78
환자E	3.55	3.91	2.68	4.22	확진	1.28
환자F	4.12	2.86	3.30	3.71	확진	1.31
새로운 환자	3.24	3.68	3.82	3.77	확진	

```
knr = KNeighborsClassifier(n_neighbors=3) #객체 생성
```

```
knr.fit(X, labels)#훈련
```

```
knr.predict(new)
```

```
array(['확진'], dtype='<U2')
```

03

k-최근접 이웃을 통한 회귀



k-최근접 이웃을 통한 회귀

- y의 값이 이산형 데이터가 아닌 연속형 데이터일 경우에는 회귀를 수행
- 알고리즘의 동작은 동일하나, k 개 데이터의 값의 평균을 회귀값으로 반환
 - 거리에 비례하여 가중치를 주는 방법도 존재

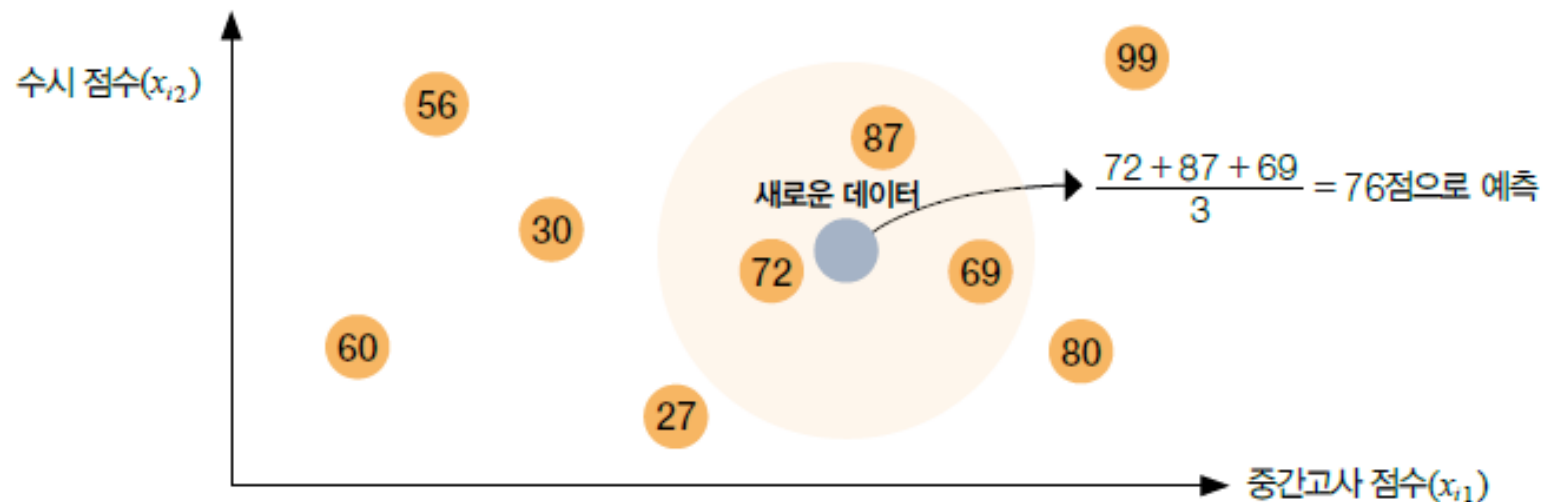


그림 7-11 k-최근접 이웃을 통한 회귀 예 ($k=3$)



k-최근접 이웃을 통한 회귀 코드

- 학생의 과목별 최종 점수와 인공지능 입문 최종 점수 정보
- 학생의 인공지능 최종 점수를 k-최근접 이웃을 통해 예측 (k=1, k=3)

표 7-5 타과목 점수에 따른 인공지능 입문 점수 데이터(회귀)

	미적분학	기초물리학	프로그래밍	통계의 이해	군사영어	인공지능 입문
학생0	7.5	7.5	7.0	9.5	8.5	5.0
학생1	7.5	7.0	7.5	8.0	8.0	6.0
학생2	8.0	7.0	8.0	8.0	8.5	8.5
학생3	8.5	8.0	9.5	7.5	6.0	7.0
학생4	10.0	9.5	9.0	7.5	7.5	10.0
학생5	9.0	9.0	8.0	8.0	8.0	9.0
새로운 학생	9.0	8.5	8.0	7.0	8.0	?

```
# 입력 데이터 (다른 과목 점수)
X = np.array([ [7.5, 7.5, 7.0, 9.5, 8.5], # 학생0
               [7.5, 7.0, 7.5, 8.0, 8.0], # 학생1
               [8.0, 7.0, 8.0, 8.0, 8.5], # 학생2
               [8.5, 8.0, 9.5, 7.5, 6.0], # 학생3
               [10.0, 9.5, 9.0, 7.5, 7.5], # 학생4
               [9.0, 9.0, 8.0, 8.0, 8.0]]) # 학생5
```

```
# 정답 (인공지능 점수)
y = np.array([5.0, 6.0, 8.5, 7.0, 10.0, 9.0])
```

```
# 신입 학생
new = np.array([9.0, 8.5, 8.0, 7.0, 8.0])
```



학습하기: 유클리드 거리 기반

표 7-6 타과목 점수에 따른 인공지능 입문 점수 데이터

	미적분학	기초물리학	프로그래밍	통계의 이해	군사영어	인공지능 입문	새 관측치와의 거리
학생0	7.5	7.5	7.0	9.5	8.5	5.0	3.28
학생1	7.5	7.0	7.5	8.0	8.0	6.0	2.40
학생2	8.0	7.0	8.0	8.0	8.5	8.5	2.12
학생3	8.5	8.0	9.5	7.5	6.0	7.0	2.65
학생4	10.0	9.5	9.0	7.5	7.5	10.0	1.87
학생5	9.0	9.0	8.0	8.0	8.0	9.0	1.12
새로운학생	9.0	8.5	8.0	7.0	8.0	?	

```
distances = np.sqrt(np.sum((X - new) ** 2, axis=1))
```

```
idx = np.argsort(distances)# 가까운 순서대로 인덱스 정렬
```

```
k=3# k=3 선택
```

```
nearY = y[idx[:k]]
```

```
print(nearY)
```

```
pred = np.mean(nearY)# 평균 (회귀 결과)
```

```
print("예측값:", pred)
```



[9. 10. 8.5]

예측값: 9.166666666666666



학습하기: 사이킷런 기반

표 7-6 타과목 점수에 따른 인공지능 입문 점수 데이터

	미적분학	기초물리학	프로그래밍	통계의 이해	군사영어	인공지능 입문	새 관측치와의 거리
학생0	7.5	7.5	7.0	9.5	8.5	5.0	3.28
학생1	7.5	7.0	7.5	8.0	8.0	6.0	2.40
학생2	8.0	7.0	8.0	8.0	8.5	8.5	2.12
학생3	8.5	8.0	9.5	7.5	6.0	7.0	2.65
학생4	10.0	9.5	9.0	7.5	7.5	10.0	1.87
학생5	9.0	9.0	8.0	8.0	8.0	9.0	1.12
새로운학생	9.0	8.5	8.0	7.0	8.0	?	

```

from sklearn.neighbors import KNeighborsRegressor
knr = KNeighborsRegressor(n_neighbors=3)
knr.fit(X, y) # 학습

# 예측
pred = knr.predict([new])
print("예측값:", pred[0])

```



예측값: 9.166666666666666

04

k-최근접 이웃의 하이퍼파라미터



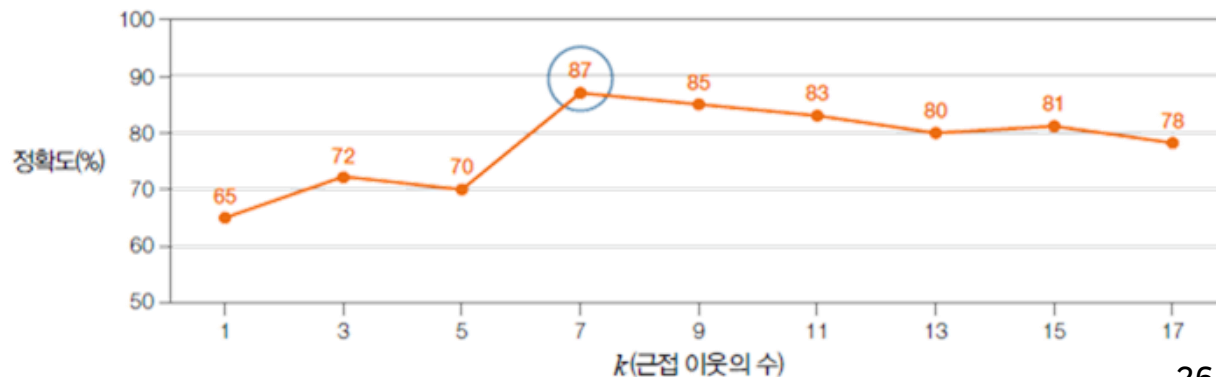
KNN의 장단점

- K-최근접 이웃(KNN) 장점
 - 직관적이고 이해하기 쉬움
 - 학습 과정이 거의 없음 (lazy learning)
 - 데이터 분포 가정이 필요 없음 (비모수 방법)
 - 분류 / 회귀 모두 적용 가능
- K-최근접 이웃(KNN) 단점
 - 모든 데이터와 거리를 계산하기 때문에 예측 시 계산량이 큼
 - 거리 기준에 민감 (스케일 영향 큼 → 정규화 필요)
 - k 값에 따라 성능 크게 달라짐
 - 차원이 높아지면 성능 저하 (차원의 저주)
 - 노이즈 데이터에 민감



k-최근접 이웃의 하이퍼파라미터

- 하이퍼파라미터
 - 학습을 통해 탐색되지 않는 사용자 직접 결정 파라미터
 - 적절한 값 설정이 모델 성능에 큰 영향을 미침
- k 값의 역할: k는 가장 가까운 데이터의 개수로 모델의 성능에 중요한 영향을 미침
- k 값 설정의 문제
 - 작은 k 값: 데이터의 지역적 특성을 과도하게 반영 => 과대 적합 유발
 - 큰 k 값: 다른 범주의 개체 포함, 잘못 분류 위험 => 과소 적합 유발
- 적절한 k 값 찾기
 - k 값을 변경하며 성능을 평가
 - 가장 좋은 성능을 보일 때의 k를 선택





데이터 불러오기

- 농구 선수의 기록 기반 데이터셋에서 3점슛 성공 수(3P)와 블록슛 수(BLK)를 기반으로 포지션을 예측

컬럼명	설명
Unnamed: 0	선수 고유 인덱스 (ID 역할)
Player	선수 이름
Pos	포지션 (SG: 슈팅가드, C: 센터 등)
3P	3점슛 성공 수 (또는 평균)
TRB	리바운드 수 (Total Rebounds)
BLK	블록슛 수

```
import pandas as pd
train = pd.read_csv('./data/basketball_train.csv')
test = pd.read_csv('./data/basketball_test.csv')

print(train.head(3))
print(test.head(3))
```



	Unnamed: 0	Player	Pos	3P	TRB	BLK
0	26	Wayne Ellington	SG	2.4	2.1	0.1
1	86	Nik Stauskas	SG	1.7	2.8	0.4
2	60	Alex Len	C	0.0	6.6	1.3
	Unnamed: 0	Player	Pos	3P	TRB	BLK
0	96	Dwyane Wade	SG	0.8	4.5	0.7
1	39	Tim Hardaway	SG	1.9	2.8	0.2
2	21	Jordan Crawford	SG	1.9	1.8	0.1



전처리: 데이터 분리

필요한 피쳐만 불러오기

```
#train
X_train = train[['3P', 'BLK']].values
y_train = train[['Pos']].values

#test
X_test = test[['3P', 'BLK']].values
y_test = test[['Pos']].values

X_train.shape, X_test.shape
```



`((80, 2), (20, 2))`

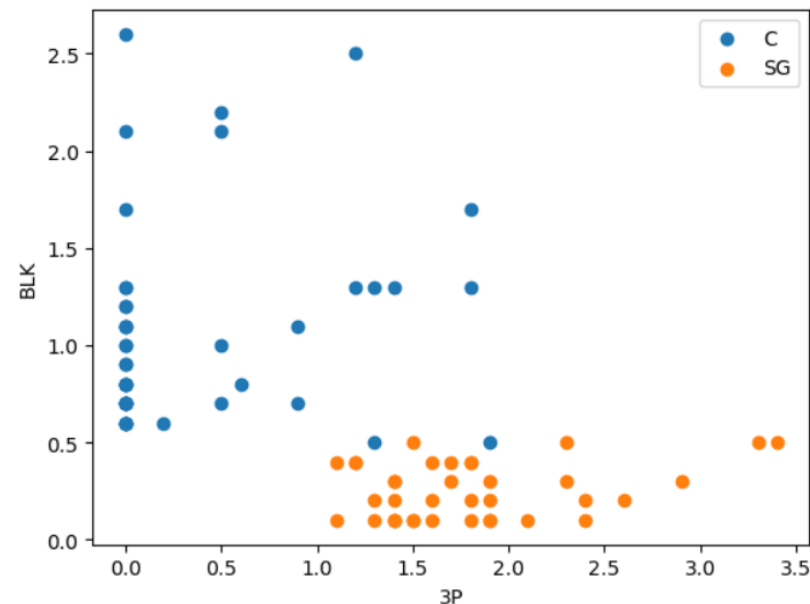
훈련 데이터셋 시각화

```
import matplotlib.pyplot as plt

label = np.unique(y_train)

for p in label:
    idx = np.where(y_train == p)[0]
    plt.scatter(X_train[idx,0], X_train[idx,1], label = p)

plt.xlabel("3P") # x축 표시
plt.ylabel("BLK") # y축 표시
plt.legend()
plt.show()
```





최적의 k값 찾기

- k를 점차 증가 시키면서 처음으로 가장 좋은 성능을 보이는 k값 선택

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

accuracy = []
k = np.arange(1,11)#k=1~10

for i in k:
    kNN = KNeighborsClassifier(n_neighbors = i)
    kNN.fit(X_train, y_train)
    y_pred = kNN.predict(X_test)
    accuracy.append(accuracy_score(y_test, y_pred))

print(accuracy)
print(np.argmax(accuracy)+1)
```

```
[0.8, 0.75, 0.9, 0.9, 0.9, 0.9, 0.9, 0.9, 0.9, 0.9]
```

```
3
```



최적의 k값(=3)으로 학습

#최적의 k값으로 학습

```
kNN = KNeighborsClassifier(n_neighbors = 3)  
kNN.fit(X_train, y_train)
```

#예측값

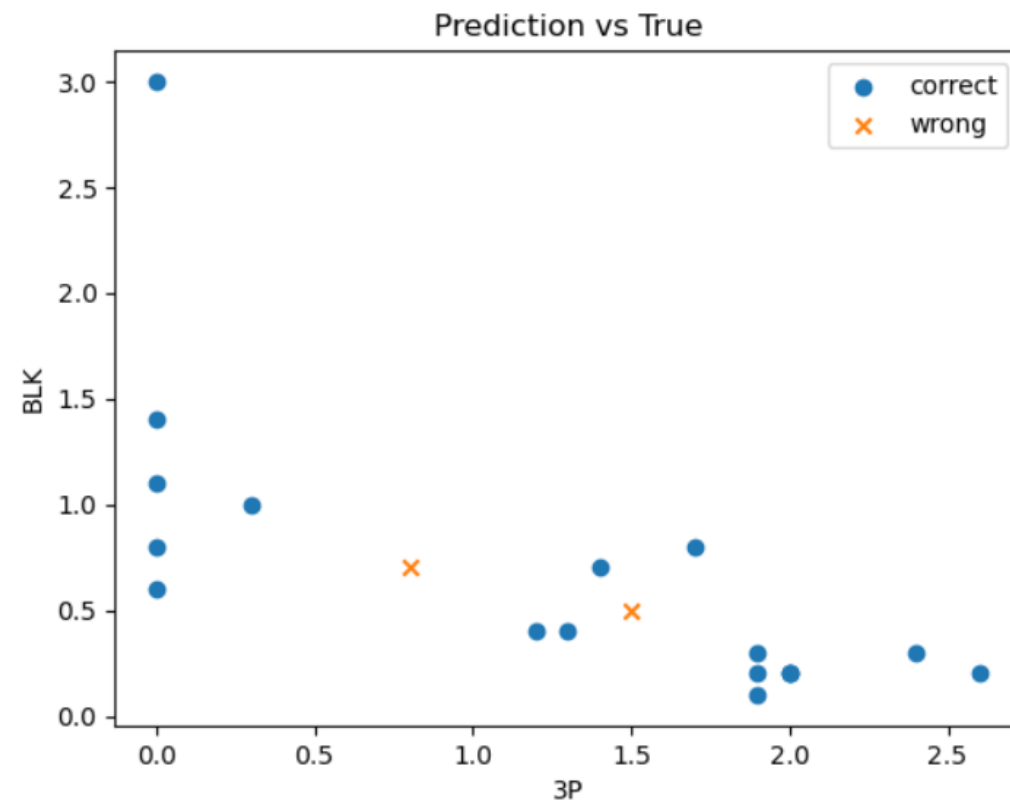
```
y_pred = kNN.predict(X_test)
```

#그래프

```
crr_idx = np.where(y_test== y_pred)  
wr_idx = np.where(y_test!= y_pred)
```

```
plt.scatter(X_test[crr_idx,0], X_test[crr_idx,1],  
            marker='o', label='correct')  
plt.scatter(X_test[wr_idx,0], X_test[wr_idx,1],  
            marker='x', label='wrong')
```

```
plt.xlabel("3P")  
plt.ylabel("BLK")  
plt.legend()  
plt.show()
```





성능평가

```
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

```
[[ 6  1]
 [ 1 12]]
```

	precision	recall	f1-score	support
C	0.86	0.86	0.86	7
SG	0.92	0.92	0.92	13
accuracy			0.90	20
macro avg	0.89	0.89	0.89	20
weighted avg	0.90	0.90	0.90	20



SUMMARY

- 모델 기반 학습과 사례 기반 학습의 두 방식의 차이
 - 모델을 학습하는 방식 vs 데이터를 그대로 활용하는 방식
- k-최근접 이웃(KNN)의 기본 개념과 동작 원리
 - 새로운 데이터가 주어졌을 때, 가장 가까운 k개의 이웃을 기반으로 예측 수행
- 데이터 간 유사도를 측정하기 위한 거리 기반 방법: 유클리디안 거리와 맨하튼 거리
- KNN 알고리즘을 활용하여 분류(Classification): 다수결로 결정
- KNN 알고리즘을 활용하여 회귀(Regression) : 평균값으로 결정
- 다양한 k값에 따른 성능 변화를 비교하고 최적의 k값을 선택
 - 정확도 기반 평가를 통해 최적의 모델 선정



Q&A

궁금한 사항은 아래 방법으로 문의 바랍니다.



이메일
dana1112@bible.ac.kr



LMS 쪽지

THANK YOU FOR YOUR ATTENTION!

한국성서대학교 | 인공지능융합학부 | 머신러닝